

Finding Undervalued NBA Players

Question: Which players produce more than their contract pays for, and which of those could a team realistically sign at a discount?

Approach: Scrape 2025-26 per-game and advanced stats from Basketball Reference, join them to 2026-27 salaries from the contracts page, and fit a model of what the market pays for a given statistical profile. A player whose actual salary sits well below what the model expects for his production is a candidate. The model is not a claim about what a player is worth in wins. It is a description of how this league currently prices production, and the residual from it is the signal.

Why 2025-26 stats against 2026-27 salary? Salary is set at signing, not earned in-season, so regressing this year's pay on this year's production mostly recovers decisions made two or three offseasons ago. Pairing last season's production with next season's pay asks the forward-looking question instead.

Scope: One season. Basketball Reference exposes bulk salary only for the current contract year on </contracts/players.html>; historical per-season salary lives on individual player pages, which would mean roughly 600 requests per season against a 20-request-per-minute rate limit. A single season also removes the salary cap inflation confound entirely.

The salary year is pinned. If Basketball Reference rolls the contracts page forward, the notebook raises rather than quietly retargeting a later season.

```
import sys
!{sys.executable} -m pip install -q pandas numpy scikit-learn xgboost lxml requests matpl
```

[notice] A new release of pip is available: 23.1.2 -> 26.1.2

[notice] To update, run: pip3.11 install --upgrade pip

```
import io
import re
import time
import unicodedata

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import requests
from scipy import stats
from sklearn.base import clone
from sklearn.inspection import permutation_importance
from sklearn.model_selection import (GridSearchCV, KFold, cross_val_predict,
                                     learning_curve, train_test_split)
from sklearn.metrics import mean_absolute_error, r2_score
from xgboost import XGBRegressor
```

```

SEASON_END_YEAR = 2026          # 2026 means the 2025–26 season, the last completed one
SALARY_SEASON = "2026–27"      # pinned; the contracts page rolls forward, this must not
HEADERS = {"User-Agent": "Mozilla/5.0"}
BASE = "https://www.basketball-reference.com"

```

```

def read_bbref_table(url, table_id):
    """Fetch a Basketball Reference page and return one table as a DataFrame.

    Two details matter here. The response headers omit a charset, so requests falls
    back to ISO-8859-1 and mangles accented names (Vucevic, Diabate, Jokic); the
    encoding is set explicitly. And some tables are wrapped in HTML comments to
    defer rendering, so the comment markers are stripped before parsing.
    """
    resp = requests.get(url, headers=HEADERS, timeout=30)
    resp.encoding = "utf-8"
    html = resp.text.replace("<!--", "").replace("-->", "")
    tables = pd.read_html(io.StringIO(html), attrs={"id": table_id})
    time.sleep(3)
    return tables[0]

```

```

per_game = read_bbref_table(f"{BASE}/leagues/NBA_{SEASON_END_YEAR}_per_game.html", "per_g
advanced = read_bbref_table(f"{BASE}/leagues/NBA_{SEASON_END_YEAR}_advanced.html", "advan
contracts = read_bbref_table(f"{BASE}/contracts/players.html", "player-contracts")

print(per_game.shape, advanced.shape, contracts.shape)

```

(734, 31) (734, 29) (497, 10)

```

def normalize_name(name):
    """Lowercase, drop accents, punctuation, and Jr./II/III style suffixes."""
    text = unicodedata.normalize("NFKD", str(name))
    text = "".join(ch for ch in text if not unicodedata.combining(ch)).lower()
    text = re.sub(r"[\.\'\`\-]", "", text)
    text = re.sub(r"^[a-z0-9 ]", " ", text)
    drop = {"jr", "sr", "ii", "iii", "iv", "v"}
    return " ".join(p for p in text.split() if p and p not in drop)

```

```

def clean_stats(df):
    """Drop repeated header rows, keep the season-total row for traded players,
    and coerce stat columns to numeric."""
    df = df.copy()
    df.columns = [str(c) for c in df.columns]
    df = df.rename(columns={"Tm": "Team"})
    df = df[df["Player"].notna() & (df["Player"] != "Player")]

    # Traded players get one row per team plus a combined row labelled 2TM/3TM

```

```
# (older seasons use TOT). Keep the combined row where it exists.
df["_key"] = df["Player"].map(normalize_name)
df["_combined"] = df["Team"].astype(str).str.match(r"^(TOT|\dTM)$")
df = (df.sort_values("_combined", ascending=False)
      .drop_duplicates("_key", keep="first")
      .drop(columns=["_combined"]))

for col in df.columns:
    if col not in {"Player", "Team", "Pos", "Awards", "_key"}:
        df[col] = pd.to_numeric(df[col], errors="coerce")
return df
```

```
def clean_contracts(df, season=SALARY_SEASON):
```

```
    """Flatten the two-row header and pull out one specific contract year.
```

```
Basketball Reference rolls this page forward as new seasons begin. The season
is pinned rather than inferred so the target cannot silently drift to a later
year; if the pinned column is gone, this raises instead of substituting one.
    """
```

```
df = df.copy()
if isinstance(df.columns, pd.MultiIndex):
    df.columns = [c[-1] for c in df.columns]
df.columns = [str(c) for c in df.columns]
df = df[df["Player"].notna() & (df["Player"] != "Player")]

year_cols = sorted(c for c in df.columns if re.fullmatch(r"\d{4}-\d{2}", c))
print(f"Salary years on the page: {year_cols}")
if season not in year_cols:
    raise ValueError(
        f"Pinned salary season {season} is not on the page. Available: {year_cols}"
    )
print(f"Using salary column: {season} (pinned)")

df["salary"] = (df[season].astype(str)
                .str.replace(r"[$,]", "", regex=True)
                .pipe(pd.to_numeric, errors="coerce"))

# Players traded midseason appear once per team; keep one row each.
df["_key"] = df["Player"].map(normalize_name)
out = (df.dropna(subset=["salary"])
      .sort_values("salary", ascending=False)
      .drop_duplicates("_key", keep="first")[["_key", "salary"]])
out.attrs["season"] = season
return out
```

```
pg = clean_stats(per_game)
adv = clean_stats(advanced)
sal = clean_contracts(contracts)
```

```
# Advanced adds PER, TS%, BPM, VORP etc. Drop the columns it duplicates from per-game.
overlap = [c for c in adv.columns if c in pg.columns and c != "_key"]
data = (pg.merge(adv.drop(columns=overlap), on="_key", how="inner")
        .merge(sal, on="_key", how="inner"))

# Salary is only meaningful for players with a real role; very low game counts
# are mostly two-way and 10-day contracts that the stats cannot explain.
data = data[data["G"] >= 20].reset_index(drop=True)

# Rookie scale contracts are fixed by the CBA rather than negotiated, so the model
# is told which players are on one instead of being left to average the two regimes
# together. Derived from age: a flag built from the salary itself would be a
# deterministic function of the target and would leak.
ROOKIE_SCALE_MAX_AGE = 24
data["rookie_scale"] = (data["Age"] <= ROOKIE_SCALE_MAX_AGE).astype(int)

stats_season = f"{SEASON_END_YEAR - 1}-{str(SEASON_END_YEAR)[2:]}"
print(f"\n{len(data)} players: {stats_season} stats against {sal.attrs['season']} salary")
print(f"{data['rookie_scale'].sum()} on rookie scale (proxy: age {ROOKIE_SCALE_MAX_AGE} or under), {data['rookie_scale'].sum()} veterans")
data[["Player", "Team", "Pos", "G", "PTS", "PER", "salary"]].head()
```

Salary years on the page: ['2026-27', '2027-28', '2028-29', '2029-30', '2030-31', '2031-32']

Using salary column: 2026-27 (pinned)

370 players: 2025-26 stats against 2026-27 salary

159 on rookie scale (proxy: age 24 or under), 211 veterans

	Player	Team	Pos	G	PTS	PER	salary
0	Tyus Jones	3TM	PG	67.0	3.0	8.3	2449421
1	Walter Clayton	2TM	PG	69.0	7.8	10.3	4190520
2	AJ Johnson	2TM	SG	48.0	3.3	5.6	3237120
3	CJ McCollum	2TM	PG	76.0	18.7	15.9	21000000
4	Khris Middleton	2TM	SF	63.0	10.2	10.9	5591122

The sample lands around 370 players, down from roughly 730 who appeared in a game. Most of that attrition is not the games filter. It is players with no 2026-27 salary on the page at all: expiring contracts, unsigned free agents, and two-way deals. That is a meaningful selection effect and it cuts in a specific direction. Players whose next-season pay is unknown are disproportionately marginal or between contracts, so the sample tilts toward established players on multi-year deals.

Team reads **2TM** or **3TM** for players traded midseason, which is the combined season-total row rather than any one club.

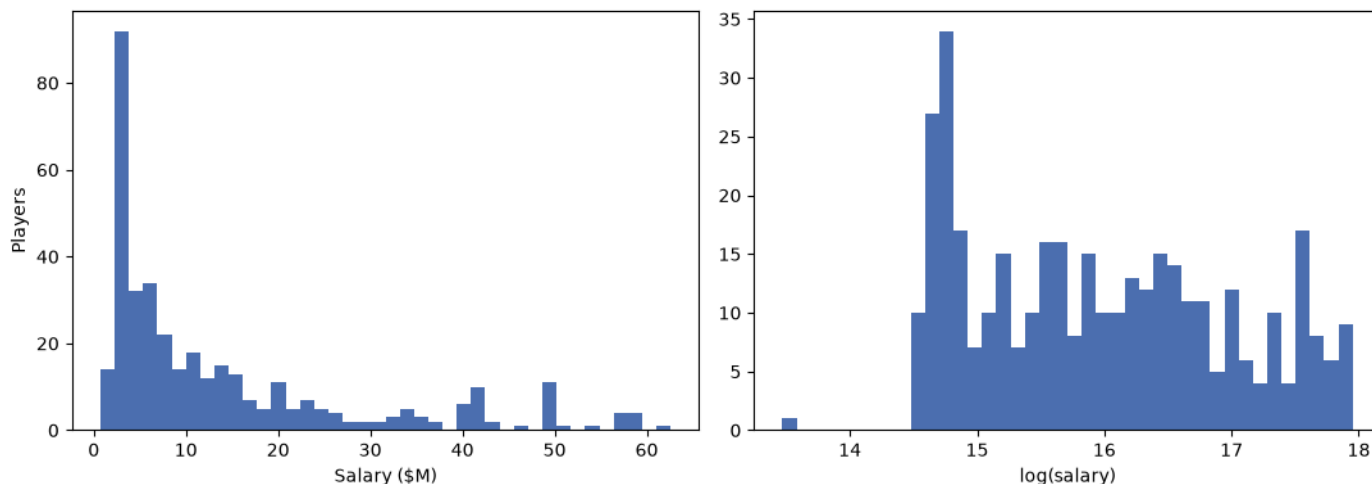
Exploring the data

```
fig, axes = plt.subplots(1, 2, figsize=(11, 4))

axes[0].hist(data["salary"] / 1e6, bins=40, color="#4c72b0")
axes[0].set_xlabel("Salary ($M)")
axes[0].set_ylabel("Players")

axes[1].hist(np.log(data["salary"]), bins=40, color="#4c72b0")
axes[1].set_xlabel("log(salary)")

plt.tight_layout()
plt.show()
```

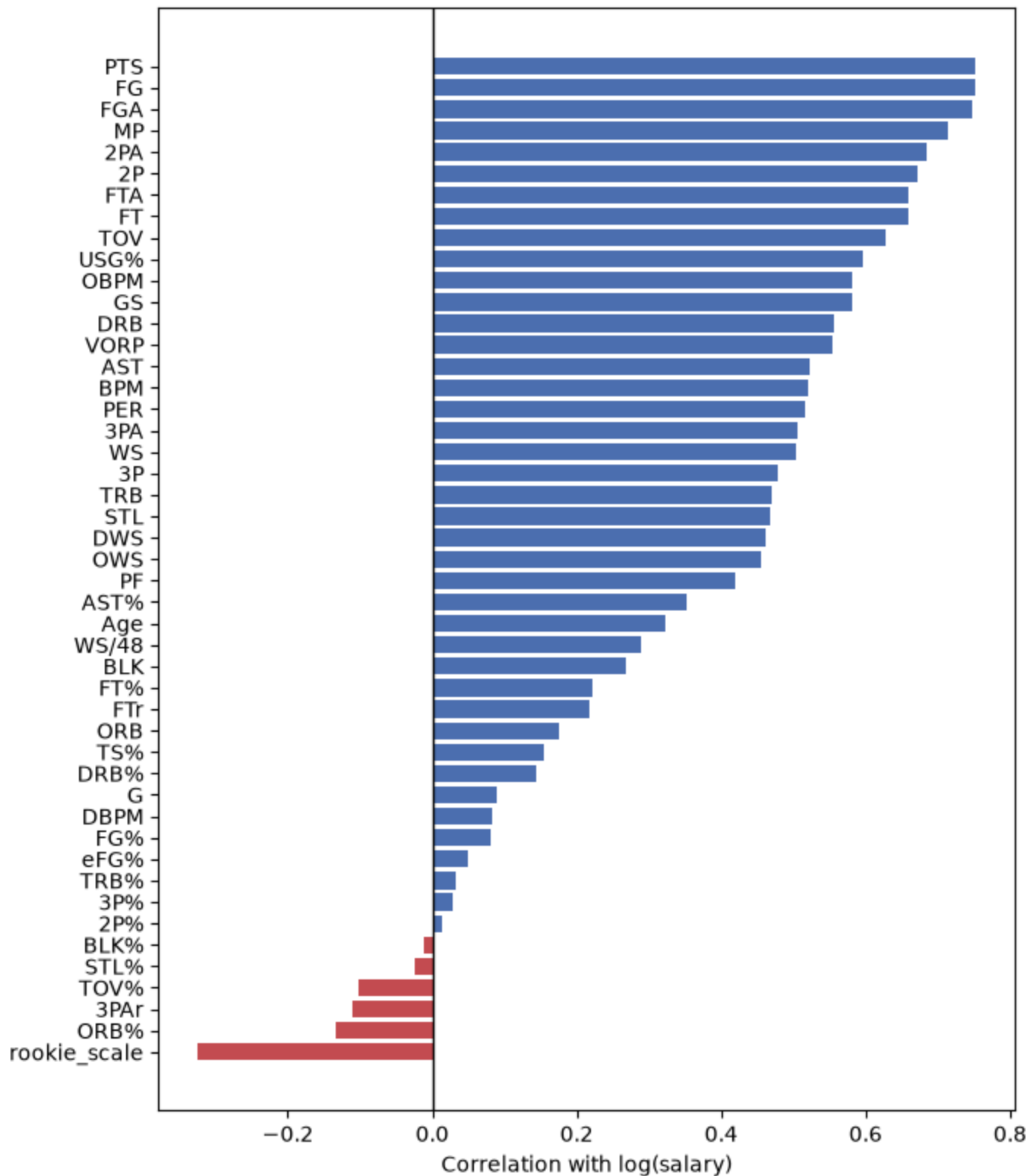


Raw salary is severely right skewed. The bulk of the league sits under \$10M with a thin tail running past \$60M, so squared error on raw dollars would be dominated almost entirely by a handful of max contracts. On the log scale the distribution flattens out across roughly 14.5 to 18 and becomes usable as a regression target.

The spike at the left edge of the logged panel is the veteran minimum. That is a genuine point mass, not a tail artifact: a large group of players are paid exactly the same number regardless of what they produced. No transformation removes it, and it sets a floor the model has to fit around.

```
numeric = data.select_dtypes("number").drop(columns=["Rk"], errors="ignore")
numeric = numeric.loc[:, numeric.std() > 0] # constant columns have undefined co
corr = numeric.corrwith(np.log(data["salary"])).drop("salary").dropna().sort_values()

fig, ax = plt.subplots(figsize=(7, 8))
ax.barh(corr.index, corr.values, color=np.where(corr.values > 0, "#4c72b0", "#c44e52"))
ax.axvline(0, lw=1, color="black")
ax.set_xlabel("Correlation with log(salary)")
plt.tight_layout()
plt.show()
```



This is the most important plot in the notebook and it largely determines what “undervalued” can mean here.

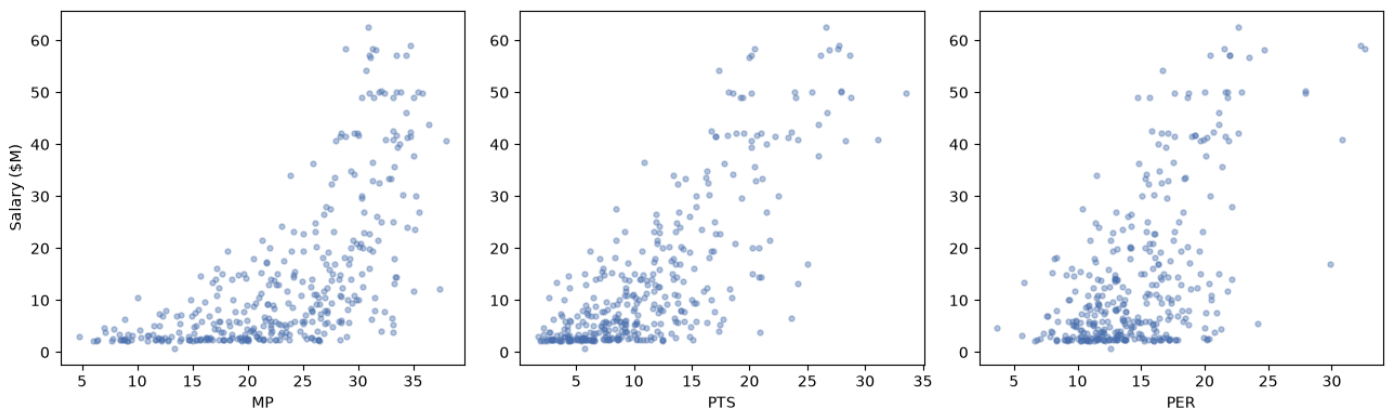
Everything at the top is volume: points, field goals, attempts, minutes, free throw attempts, turnovers. These correlate with pay around 0.6 to 0.75. Everything measuring efficiency sits near zero: true shooting around 0.16, effective field goal percentage near 0.05, field goal percentage near 0.08. Three of the four negative correlations are rate statistics.

The league pays for role and usage, not for efficiency. A model fit on these features will therefore identify as underpaid the players who absorb minutes and shots without a matching salary, and it will say almost nothing about players who are efficient in a small role. That is a property of the market being described, not a flaw in the fit, but it bounds what the output can be used for.

```
fig, axes = plt.subplots(1, 3, figsize=(13, 4))

for ax, col in zip(axes, ["MP", "PTS", "PER"]):
    if col in data.columns:
        ax.scatter(data[col], data["salary"] / 1e6, s=12, alpha=0.4, color="#4c72b0")
        ax.set_xlabel(col)
axes[0].set_ylabel("Salary ($M)")

plt.tight_layout()
plt.show()
```



Three features worth noting.

The relationships are not linear. Salary stays flat and low across a wide band of production, then turns upward sharply past a threshold, which is roughly where a player stops being a rotation piece and starts being paid as a starter or better.

Variance grows with production. Low-production players are all paid about the same because they are all on minimums, while high-production players are spread across \$10M to \$60M. The spread is multiplicative, which is a second argument for the log target.

There is a hard floor visible across the entire x range and a ceiling near \$60M. Both are CBA artifacts rather than market outcomes.

Rate normalization

Before choosing a model, one transformation is worth making, because most of the correlation between box score columns is not basketball. It is shared playing time. Points, field goals, attempts, free throws and rebounds all move together largely because they all measure “this player is on the floor.”

Dividing each counting stat by minutes and rescaling to a per-36 basis splits that into two separate questions. Minutes carries the role signal on its own, and the rate columns carry productivity. The two are close to independent, which is both a cleaner model input and a sharper economic question: does the market pay for playing time, or for what a player does with it?

```
COUNTING_STATS = ["FG", "FGA", "3P", "3PA", "2P", "2PA", "FT", "FTA",
                  "ORB", "DRB", "TRB", "AST", "STL", "BLK", "TOV", "PF", "PTS"]

# Exact arithmetic functions of other columns. Rate normalization does not break
# these: dividing both sides of  $PTS = 2*2P + 3*3P + FT$  by minutes leaves the
# identity intact, so they still have to be removed by hand.
IDENTITY_COLS = {"PTS", "FG", "FGA", "TRB", "WS", "BPM", "VORP", "WS/48"}

def to_per36(df, per=36.0, minutes_col="MP", suffix="_p36"):
    """Replace counting stats with a per-36-minute rate, keeping minutes separate."""
    out = df.copy()
    minutes = out[minutes_col].replace(0, np.nan)
    for col in COUNTING_STATS:
        if col in out.columns:
            out[f"{col}{suffix}"] = out[col] / minutes * per
            out = out.drop(columns=[col])
    return out

rated = to_per36(data)
y = np.log(data["salary"])

print(f"{data.shape[1]} columns in, {rated.shape[1]} out")
print("Rate columns:", ", ".join(c for c in rated.columns if c.endswith("_p36")))
```

54 columns in, 54 out

Rate columns: FG_p36, FGA_p36, 3P_p36, 3PA_p36, 2P_p36, 2PA_p36, FT_p36, FTA_p36, ORB_p36, DRB_p36, TRB_p36, AST_p36, STL_p36, BLK_p36, TOV_p36, PF_p36, PTS_p36

```
pairs = [(c, f"{c}_p36") for c in COUNTING_STATS
          if c in data.columns and f"{c}_p36" in rated.columns]

compare = pd.DataFrame(
    {"Raw total": [data[a].corr(y) for a, _ in pairs],
     "Per 36": [rated[b].corr(y) for _, b in pairs]},
    index=[a for a, _ in pairs],
).sort_values("Raw total")

pos = np.arange(len(compare))
fig, ax = plt.subplots(figsize=(8, 7))
ax.barh(pos + 0.2, compare["Raw total"], height=0.4, label="Raw total", color="#4c72b0")
ax.barh(pos - 0.2, compare["Per 36"], height=0.4, label="Per 36", color="#dd8452")
ax.axvline(0, lw=1, color="black")
```

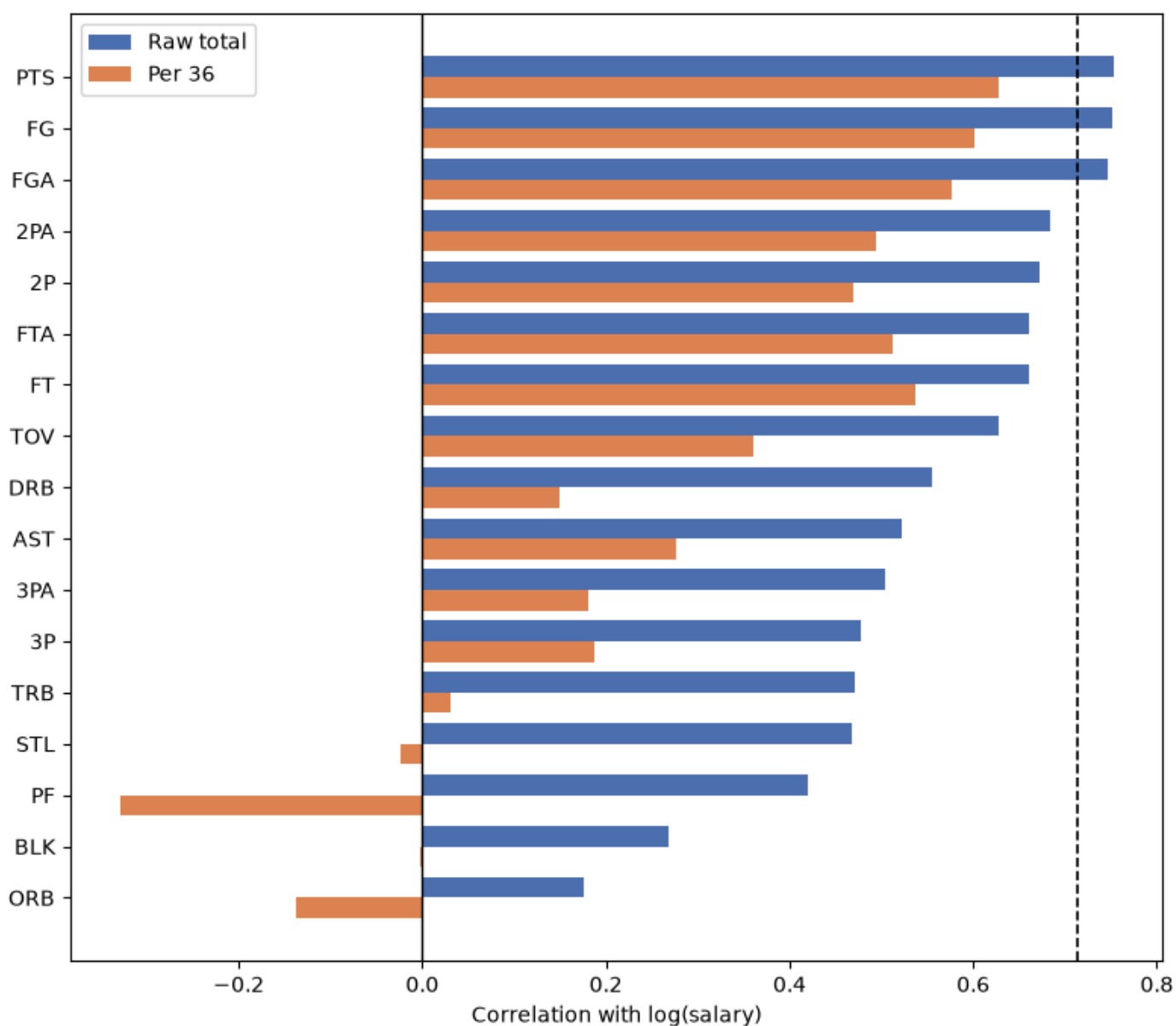


```

ax.axvline(data["MP"].corr(y), ls="--", lw=1.2, color="black")
ax.set_yticks(pos)
ax.set_yticklabels(compare.index)
ax.set_xlabel("Correlation with log(salary)")
ax.legend()
plt.tight_layout()
plt.show()

raw_corr = data[[a for a, _ in pairs]].corr().abs()
p36_corr = rated[[b for _, b in pairs]].corr().abs()
off = lambda m: m.values[~np.eye(len(m), dtype=bool)].mean()
print(f"Mean |r| between counting stats, raw: {off(raw_corr):.3f}")
print(f"Mean |r| between counting stats, per 36: {off(p36_corr):.3f}")
print(f"MP on its own correlates {data['MP'].corr(y):.3f} with log(salary)")

```



Mean |r| between counting stats, raw: 0.533

Mean |r| between counting stats, per 36: 0.350

MP on its own correlates 0.714 with $\log(\text{salary})$

The dashed vertical line is how well minutes alone predicts salary. Read every bar against it.

Wherever an orange bar sits far to the left of its blue counterpart, that statistic was borrowing its apparent relationship with salary from playing time rather than earning it. The gap between the pair is the share of the raw correlation that was really just minutes.

The printed mean absolute correlation between counting stats before and after is the collinearity payoff, and it is usually large. That does not improve accuracy much, since trees were already handling the redundancy, but it means the importances downstream describe distinguishable things.

The substantive reading is the comparison against the dashed line. If the per-36 bars sit well below it, this market prices playing time more than per-minute production, and being underpaid mostly means absorbing a starter's workload on a reserve's deal. If several per-36 bars clear it, there is genuine pricing of rate production and the bargain hunt has more to work with.

Choosing a model

The exploration above points to four constraints, and they narrow the choice considerably.

Nonlinear and threshold-shaped. Pay is flat across a wide band of production then rises steeply. A linear model would have to average through that and would misprice both ends.

Bounded on both sides. The minimum salary is a floor and the max contract is a ceiling, neither of which is a smooth function of production. Trees represent flat regions and sharp breaks natively.

Heavily collinear features. Points, field goals, attempts and minutes are near substitutes for one another. Ordinary least squares would produce unstable, uninterpretable coefficients here. Tree ensembles are unaffected in their predictions, though as shown below their attribution is not.

Small sample. Around 370 players and more than 40 candidate features. That argues for heavy regularization and cross validation rather than a single fit, and against anything with many parameters.

Gradient boosted trees with a grid search over depth, learning rate and subsampling fit those constraints. The target is $\log(\text{salary})$, which handles the skew and makes the error multiplicative: being wrong by \$2M on a \$4M player and by \$20M on a \$40M player count the same, which matches how salaries actually scale.

```
def prune_collinear(X, y, threshold=0.9):
    """Greedy keep features, preferring those most correlated with the target,
    and drop any that duplicate a feature already kept."""
    X = X.loc[:, X.std(numeric_only=True) > 0]      # constant columns carry no signal
    corr = X.corr().abs()
    ranked = X.corrwith(y).abs().sort_values(ascending=False).index
    keep = []
    for col in ranked:
        if all(corr.loc[col, k] < threshold for k in keep):
```

```

        keep.append(col)
    return keep

```

```

drop_cols = {"Rk", "Player", "Team", "Awards", "_key", "salary"}

# to_per36 renames the counting stats, so the identity set has to cover both forms.
blocked = IDENTITY_COLS | {f"{c}_p36" for c in IDENTITY_COLS}

# rookie_scale is held out of the pruning entirely. It is correlated with Age by
# construction, so the rule would drop one of the two, and it is needed intact for
# the counterfactual prediction later. A designed indicator is not an incidental stat.
candidates = [c for c in rated.columns
               if c not in drop_cols and c not in blocked and c != "rookie_scale"
               and pd.api.types.is_numeric_dtype(rated[c])]

identities_removed = sorted(set(rated.columns) & blocked)
features = prune_collinear(rated[candidates], y, threshold=0.9) + ["rookie_scale"]
dropped = sorted(set(candidates) - set(features))

print(f"Removed as arithmetic identities: {' '.join(identities_removed)}")
print(f"Kept {len(features)} features, including rookie_scale")
print(f"Dropped as collinear: {' '.join(dropped)}")

```

Removed as arithmetic identities: BPM, FGA_p36, FG_p36, PTS_p36, TRB_p36, VORP, WS, WS/48
 Kept 28 features, including rookie_scale
 Dropped as collinear: 2P_p36, 3PA_p36, AST_p36, BLK_p36, DRB%, FTA_p36, ORB%, PER, STL%, TRB%, eFG%

Correlation above 0.9 means two features carry effectively the same information. Trees split the credit between such features arbitrarily depending on which one a given split happens to pick, so importance rankings across a collinear block are close to noise. Pruning does not improve predictive accuracy much, since the information is retained either way, but it makes the importances readable and it reduces the number of parameters in a sample this small.

The rule keeps whichever member of each correlated block is most associated with salary and drops the rest. Expect the volume cluster to collapse to one or two survivors.

```

X = rated[features]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
print(f"{len(features)} features, {len(X_train)} train / {len(X_test)} test")

```

28 features, 296 train / 74 test

```

param_grid = {
    "n_estimators": [200, 400, 800],
    "max_depth": [2, 3, 4],
    "learning_rate": [0.03, 0.05, 0.1],
    "subsample": [0.8, 1.0],
}

```

```

    "colsample_bytree": [0.8, 1.0],
}

cv = KFold(n_splits=5, shuffle=True, random_state=42)

search = GridSearchCV(
    XGBRegressor(random_state=42, n_jobs=-1),
    param_grid,
    cv=cv,
    scoring="neg_root_mean_squared_error",
    n_jobs=-1,
    verbose=1,
)
search.fit(X_train, y_train)

print("Best params:", search.best_params_)
print(f"CV RMSE (log salary): {-search.best_score_:.3f}")

```

Fitting 5 folds for each of 108 candidates, totalling 540 fits

```

Best params: {'colsample_bytree': 0.8, 'learning_rate': 0.03, 'max_depth': 2,
'n_estimators': 200, 'subsample': 0.8}
CV RMSE (log salary): 0.645

```

The search tends to land at or near the most regularized corner of the grid: shallow trees, a slow learning rate, and both row and column subsampling active. With fewer than 300 training rows that is the expected outcome and it is worth taking at face value. The data supports a simple model, and a deeper one would fit noise.

```

pred_log = search.best_estimator_.predict(X_test)

print(f"Test R2 (log scale): {r2_score(y_test, pred_log):.3f}")
print(f"Test MAE (dollars): ${mean_absolute_error(np.exp(y_test), np.exp(pred_log)):.0f}

```

```

Test R2 (log scale): 0.584
Test MAE (dollars): $5,705,512

```

Around 0.6 on the log scale means the model captures most of the broad structure of who gets paid and misses the rest.

The dollar error is the number to keep in view. A mean absolute error in the millions, against a median salary under \$10M, means this is not a pricing tool. It cannot tell you whether a player should make \$12M or \$16M. What it can do is rank, and separate players who are paid roughly in line with their production from players who are far off it. Treat the output as a screen that produces names to investigate, not as a valuation.

Model diagnostics

```

resid = y_test - pred_log

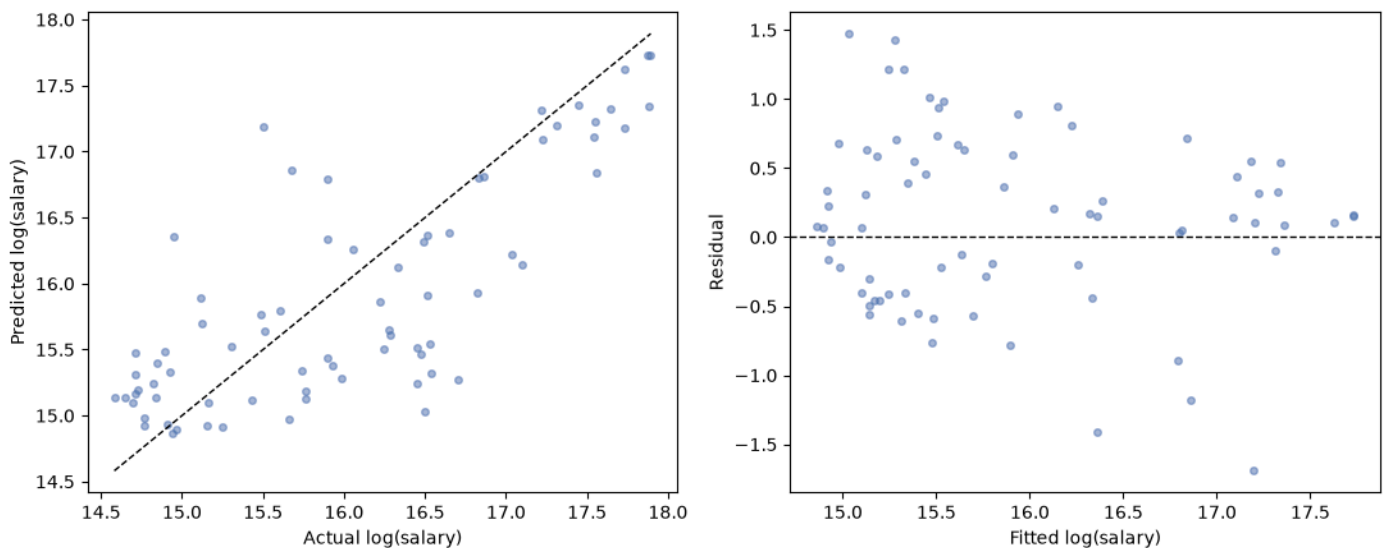
fig, axes = plt.subplots(1, 2, figsize=(11, 4.5))

axes[0].scatter(y_test, pred_log, s=16, alpha=0.5, color="#4c72b0")
lims = [min(y_test.min(), pred_log.min()), max(y_test.max(), pred_log.max())]
axes[0].plot(lims, lims, ls="--", lw=1, color="black")
axes[0].set_xlabel("Actual log(salary)")
axes[0].set_ylabel("Predicted log(salary)")

axes[1].scatter(pred_log, resid, s=16, alpha=0.5, color="#4c72b0")
axes[1].axhline(0, ls="--", lw=1, color="black")
axes[1].set_xlabel("Fitted log(salary)")
axes[1].set_ylabel("Residual")

plt.tight_layout()
plt.show()

```



The left panel shows the central problem for this use case. The cloud is flatter than the dashed identity line: at the low end predictions sit above it, at the high end they sit below. This is regression toward the mean, and it is inherent to a regularized ensemble rather than a bug.

The consequence is direct and it will show up again in the results. **Low-paid players are systematically predicted to be underpaid and high-paid players systematically predicted to be overpaid, independent of merit.** Any ranking on predicted minus actual inherits that bias, so a cheap player near the top of the undervalued list may be there partly because he is cheap.

The right panel fans out slightly, with the largest negative residuals at high fitted values. Those are players paid far above what their production predicts.

```

fig, axes = plt.subplots(1, 2, figsize=(11, 4))

axes[0].hist(resid, bins=30, color="#4c72b0")

```

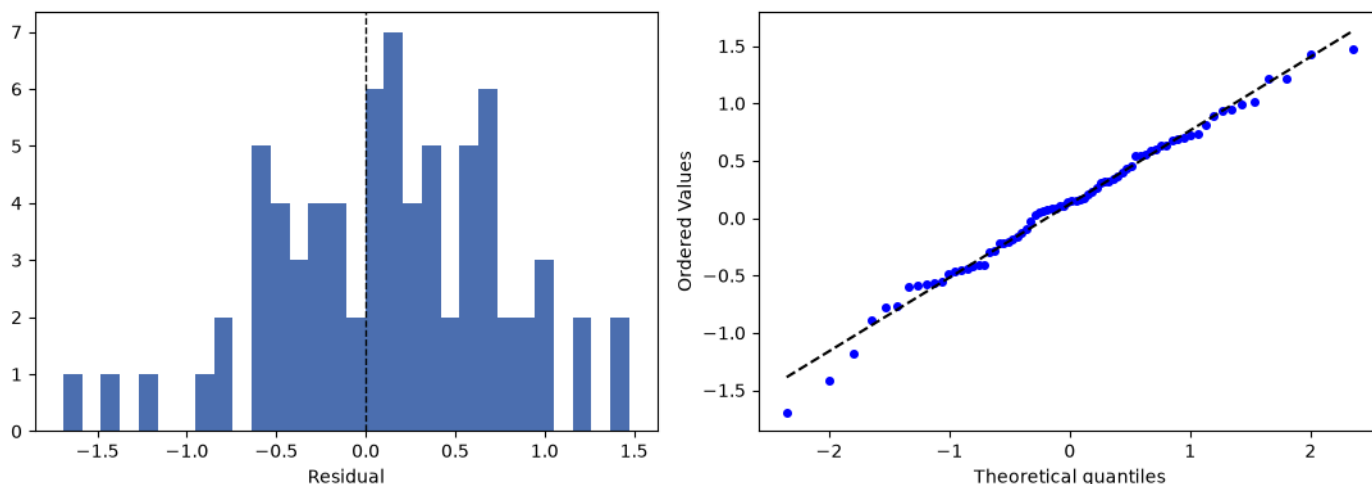
```

axes[0].axvline(0, ls="--", lw=1, color="black")
axes[0].set_xlabel("Residual")

stats.probplot(resid, dist="norm", plot=axes[1])
axes[1].get_lines()[0].set_markersize(4)
axes[1].get_lines()[1].set_linestyle("--")
axes[1].get_lines()[1].set_color("black")
axes[1].set_title("")

plt.tight_layout()
plt.show()

```



Residuals are roughly centred but carry a heavy left tail: a small group of players are paid far more than their statistics predict, with no matching group on the other side.

Gradient boosted trees carry no normality assumption, so this does not invalidate anything. It matters for one reason only. If you ever want a prediction interval around an expected salary, the normal approximation would be too narrow on the low side, and a quantile approach would be the better route.

```

perm = permutation_importance(search.best_estimator_, X_test, y_test,
                              n_repeats=20, random_state=42, n_jobs=-1)
perm_imp = pd.Series(perm.importances_mean, index=features).sort_values()

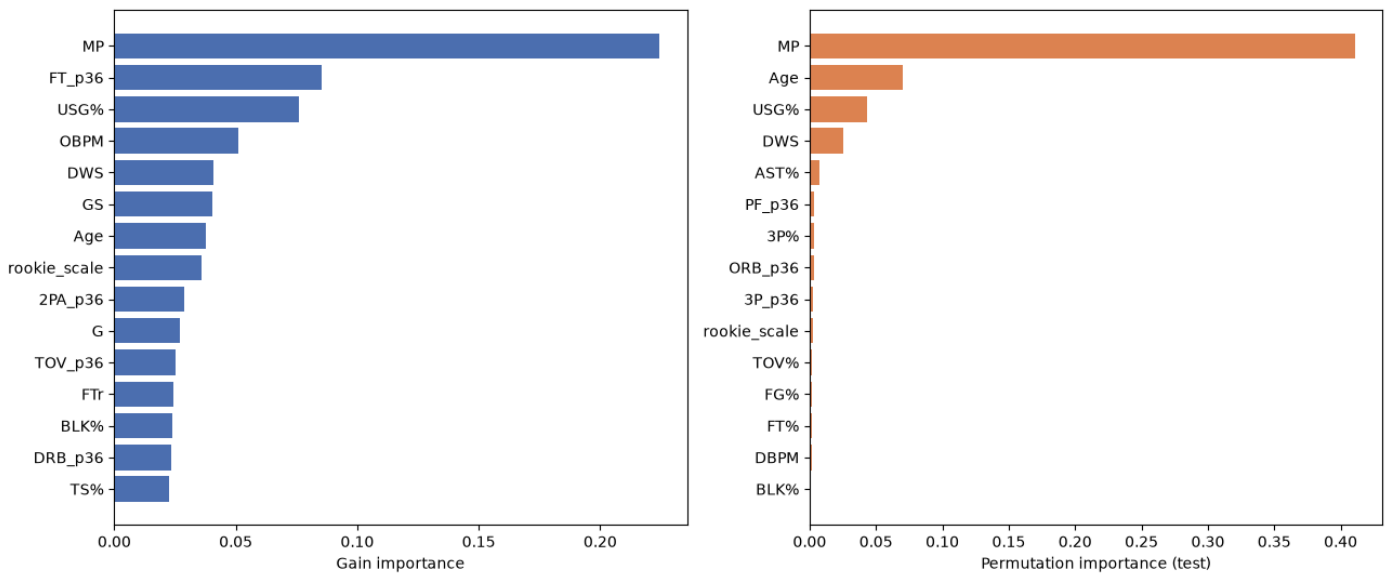
fig, axes = plt.subplots(1, 2, figsize=(13, 5.5))

gain = pd.Series(search.best_estimator_.feature_importances_, index=features).nlargest(15)
axes[0].barh(gain.index, gain.values, color="#4c72b0")
axes[0].set_xlabel("Gain importance")

top_perm = perm_imp.tail(15)
axes[1].barh(top_perm.index, top_perm.values, color="#dd8452")
axes[1].set_xlabel("Permutation importance (test)")

plt.tight_layout()
plt.show()

```



Two importance measures side by side, because they answer different questions. Gain reports how often and how usefully the training algorithm split on a feature. Permutation reports how much test error rises when that feature is shuffled, which is closer to how much the model actually depends on it out of sample.

After pruning these should broadly agree. Where they disagree, trust permutation: a feature can accumulate gain during training and still contribute nothing to held-out accuracy.

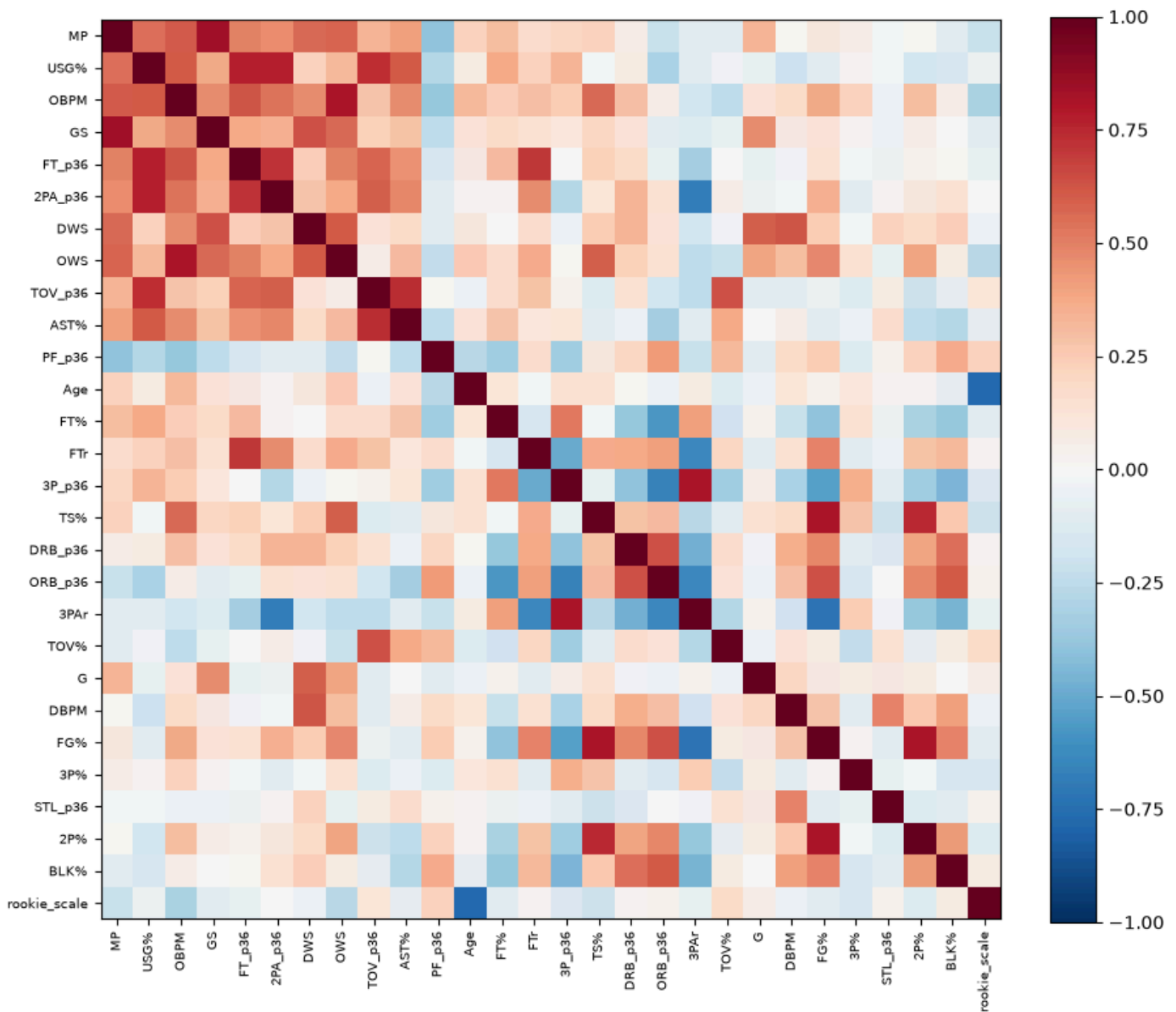
Expect volume and role features to lead either way. That is the same finding as the correlation chart, now confirmed by the fitted model rather than by marginal association.

Assumption checks

Gradient boosted trees do not carry the Gauss-Markov assumptions, so linearity, normal residuals and constant variance are not conditions for validity here. What does bear on whether this model is sound: whether it is being asked to extrapolate, whether its predictions are calibrated, whether residuals hide group structure, and whether more data would help.

```
corr_m = X.corr()

fig, ax = plt.subplots(figsize=(9, 8))
im = ax.imshow(corr_m, cmap="RdBu_r", vmin=-1, vmax=1)
ax.set_xticks(range(len(features)))
ax.set_xticklabels(features, rotation=90, fontsize=7)
ax.set_yticks(range(len(features)))
ax.set_yticklabels(features, fontsize=7)
fig.colorbar(im, fraction=0.046)
plt.tight_layout()
plt.show()
```



This is the retained feature set after pruning, so the large solid red block of near-duplicate volume statistics should be gone. Moderate correlation remains and is fine; trees handle it and the 0.9 threshold was chosen to remove only genuine redundancy rather than every relationship.

```
bins = pd.qcut(pred_log, 10, duplicates="drop")
calib = pd.DataFrame({"pred": pred_log, "actual": y_test.values}).groupby(bins, observed=

fig, axes = plt.subplots(1, 2, figsize=(11, 4.5))

axes[0].plot(calib["pred"], calib["actual"], "o-", color="#4c72b0")
lims = [min(calib.min()), max(calib.max())]
axes[0].plot(lims, lims, ls="--", lw=1, color="black")
axes[0].set_xlabel("Mean predicted log(salary)")
axes[0].set_ylabel("Mean actual log(salary)")

axes[1].hist(y_test, bins=25, alpha=0.55, label="Actual", color="#4c72b0")
axes[1].hist(pred_log, bins=25, alpha=0.55, label="Predicted", color="#dd8452")
```

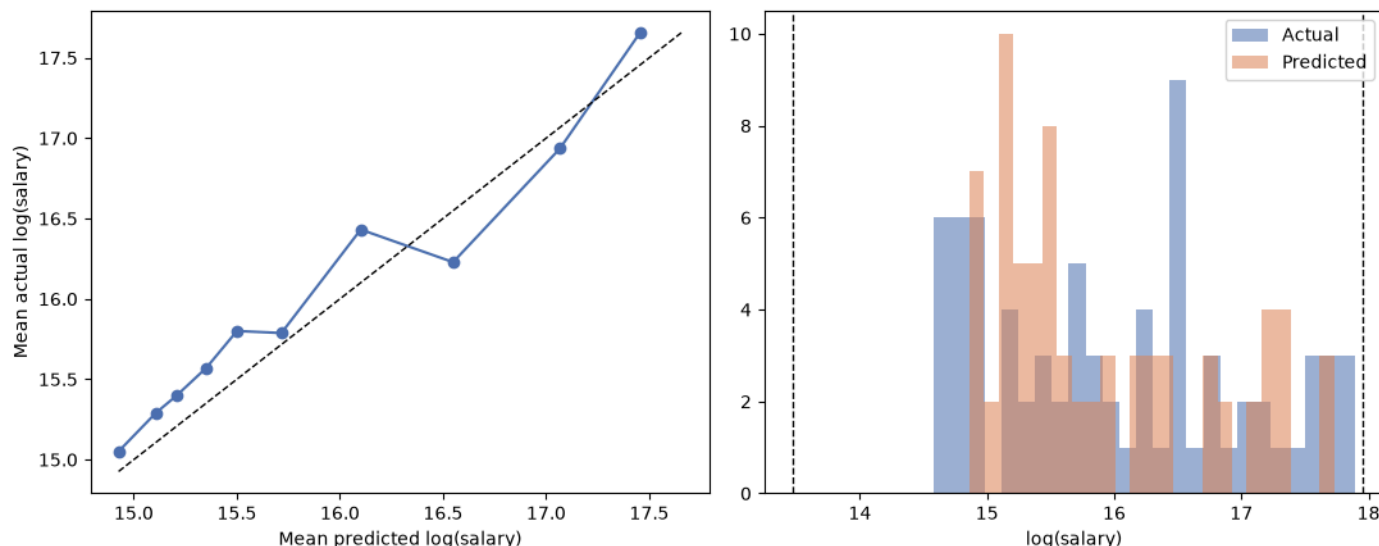


```

axes[1].axvline(y_train.min(), ls="--", lw=1, color="black")
axes[1].axvline(y_train.max(), ls="--", lw=1, color="black")
axes[1].set_xlabel("log(salary)")
axes[1].legend()

plt.tight_layout()
plt.show()

```



Left: predictions binned into deciles against mean actual salary in each bin. Deviation from the dashed line quantifies the compression seen earlier. The bottom bin sitting above the line and the top bin below it is the same regression to the mean, now measured rather than eyeballed.

Right: the predicted distribution is visibly narrower than the actual one and sits entirely inside the dashed training range. This is the extrapolation check, and it is a hard constraint rather than a tendency.

A tree ensemble cannot predict a value outside the target range it was trained on. No player will ever be assigned an expected salary above the highest salary in the training set, so every genuine superstar is structurally capped and will look overpaid.

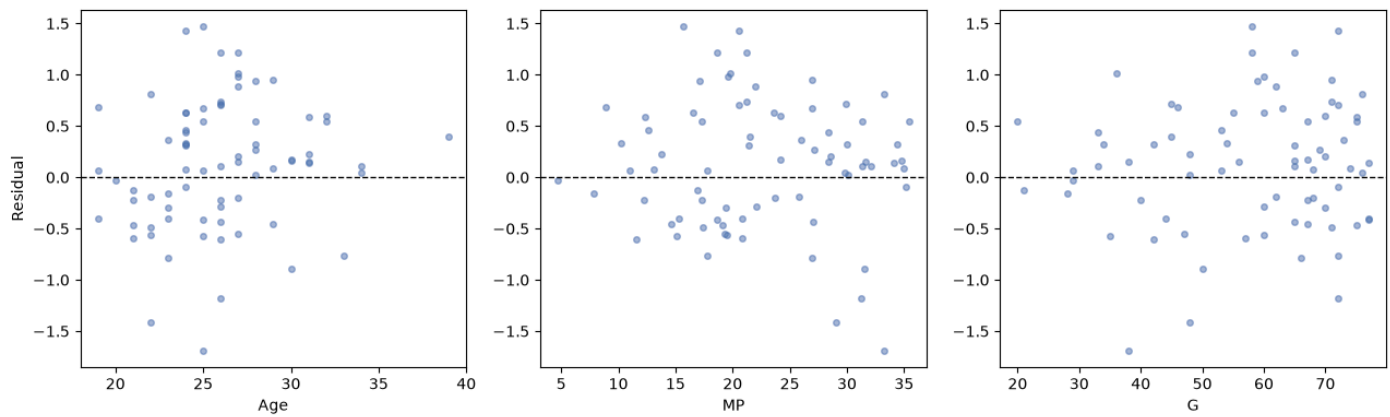
```

checks = [c for c in ["Age", "MP", "G", "PTS"] if c in X_test.columns][:3]

fig, axes = plt.subplots(1, len(checks), figsize=(4.3 * len(checks), 4))
for ax, col in zip(np.atleast_1d(axes), checks):
    ax.scatter(X_test[col], resid, s=16, alpha=0.5, color="#4c72b0")
    ax.axhline(0, ls="--", lw=1, color="black")
    ax.set_xlabel(col)
np.atleast_1d(axes)[0].set_ylabel("Residual")

plt.tight_layout()
plt.show()

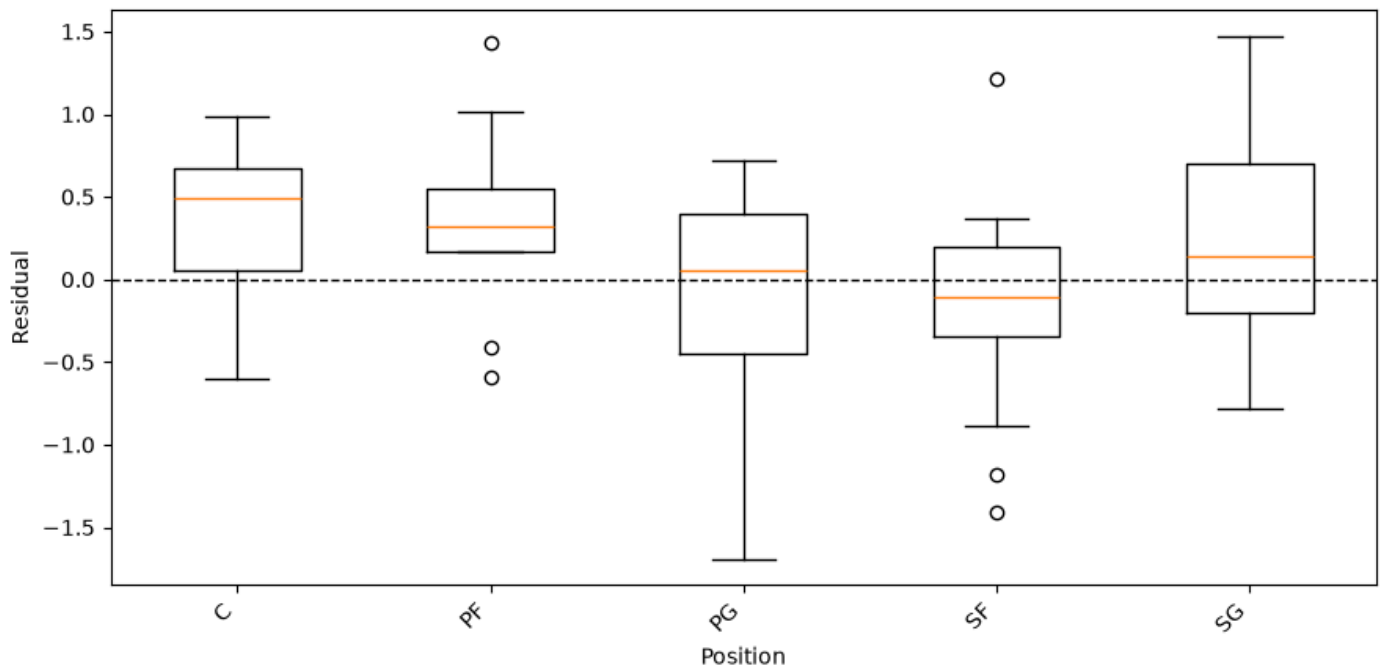
```



Residuals against features the model already uses. A visible slope would mean systematic structure left unmodelled along that dimension. Age is the one to watch: a trend there would say the model misprices players at a particular career stage, which matters directly for signing decisions.

```
groups = data.loc[X_test.index, "Pos"]
order = sorted(groups.dropna().unique())

fig, ax = plt.subplots(figsize=(9, 4.5))
ax.boxplot([resid[groups == p] for p in order])
ax.axhline(0, ls="--", lw=1, color="black")
ax.set_xticks(range(1, len(order) + 1))
ax.set_xticklabels(order, rotation=45, ha="right")
ax.set_xlabel("Position")
ax.set_ylabel("Residual")
plt.tight_layout()
plt.show()
```

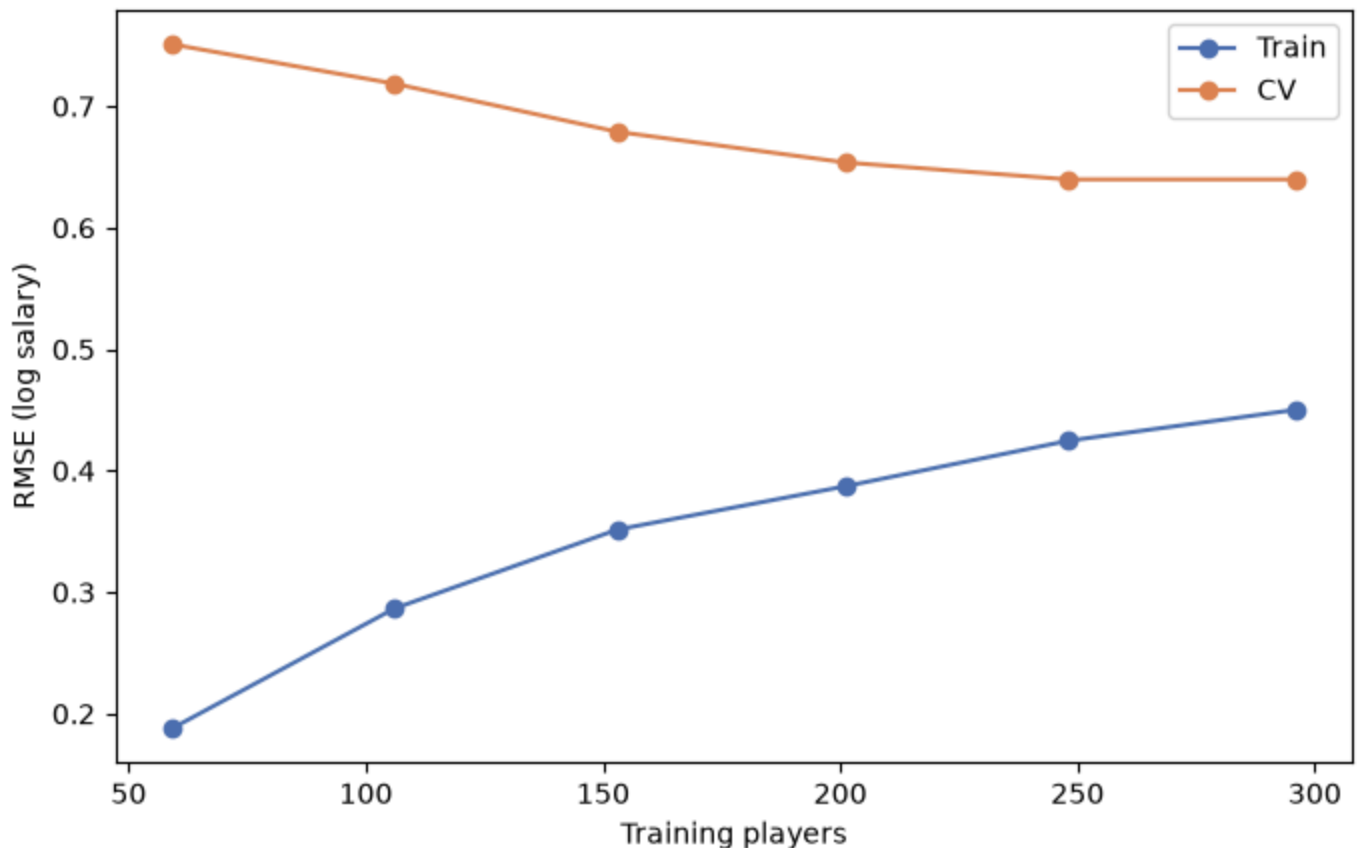


Residuals should centre on zero within every position. Where a position's box sits clearly above the line, the model under-predicts salary for that group, and those players will be over-represented on the undervalued list for a structural reason rather than a basketball one.

Bigs are the usual offender. Centres and power forwards generate their value through rebounding, rim protection and screening, and the box score captures the first of those poorly and the other two not at all. If that shows up here, adjust by comparing players within position rather than across the whole league.

```
sizes, train_scores, val_scores = learning_curve(
    search.best_estimator_, X, y, cv=cv,
    train_sizes=np.linspace(0.2, 1.0, 6),
    scoring="neg_root_mean_squared_error", n_jobs=-1,
)

fig, ax = plt.subplots(figsize=(7, 4.5))
ax.plot(sizes, -train_scores.mean(axis=1), "o-", label="Train", color="#4c72b0")
ax.plot(sizes, -val_scores.mean(axis=1), "o-", label="CV", color="#dd8452")
ax.set_xlabel("Training players")
ax.set_ylabel("RMSE (log salary)")
ax.legend()
plt.tight_layout()
plt.show()
```



The two curves converge and the validation curve flattens well before the full sample is used. That combination is informative. A small remaining gap means the model is not badly overfit, and a flat validation curve means additional players of the same kind would not help.

The model is limited by its features, not by its sample size. Adding more seasons would mostly add rows and would introduce the salary cap inflation problem. Adding tracking data, contract age, years of experience, or injury history would add information the box score does not contain.

Valuation

Every player is scored from the fold in which he was held out, so no prediction comes from a model that saw that player during training.

Because the model now knows who is on rookie scale, each player gets two numbers. **Expected** is the plain prediction, using his real contract status. **Market rate** asks the same fold's model what it would pay for that statistical profile on the open market. For a veteran the two are identical. For a rookie-scale player the gap between them is the discount his team is getting from the CBA.

Producing that second number takes more than flipping the indicator. Age and the rookie flag carry nearly the same information, so the trees lean on Age and the flag on its own barely moves the prediction. The counterfactual therefore clears the flag and raises Age to just past the cutoff, which is the minimal edit consistent with "this player is not on a rookie deal."

Surplus is market rate minus actual, so it always compares a player against open market pricing rather than against whichever regime he happens to sit in.

```
# Same folds as everywhere else, but each held-out player is scored twice: once as
# himself and once as an open market player. cross_val_predict cannot do the second
# pass, hence the explicit loop.
#
# The counterfactual has to move Age as well as the flag. Age and rookie_scale are
# strongly correlated by construction, so the trees split on Age and largely ignore
# the flag; clearing the flag alone changes nothing. Age is raised only to just past
# the rookie-scale cutoff, which is the smallest change that makes a player a
# non-rookie-scale player. Pushing it further starts pricing in the aging curve too.
X_market = X.copy()
X_market["rookie_scale"] = 0
if "Age" in X_market.columns:
    X_market["Age"] = X_market["Age"].clip(lower=ROOKIE_SCALE_MAX_AGE + 1)

oof_log = np.zeros(len(X))
market_log = np.zeros(len(X))

for train_idx, test_idx in cv.split(X):
    fold = clone(search.best_estimator_).fit(X.iloc[train_idx], y.iloc[train_idx])
    oof_log[test_idx] = fold.predict(X.iloc[test_idx])
    market_log[test_idx] = fold.predict(X_market.iloc[test_idx])
```

```

valuation = pd.DataFrame({
    "Player": data["Player"],
    "Team": data["Team"],
    "Pos": data["Pos"],
    "Age": data["Age"],
    "rookie": data["rookie_scale"],
    "G": data["G"],
    "MP": data["MP"],
    "PTS": data["PTS"],
    "actual": np.exp(y),
    "expected": np.exp(oof_log),
    "market": np.exp(market_log),
})
valuation["surplus"] = valuation["market"] - valuation["actual"]

def money(df, cols=("actual", "expected", "market", "surplus")):
    """Format dollar columns for display without needing the Styler (and jinja2)."""
    out = df.copy()
    for col in cols:
        if col in out.columns:
            out[col] = out[col].map("${:,.0f}".format)
    return out

rookies = valuation[valuation["rookie"] == 1]
print(f"Out-of-fold R2 (log scale): {r2_score(y, oof_log):.3f}")
print(f"Median CBA discount on rookie deals: "
      f"${(rookies['market'] - rookies['expected']).median():,.0f}")
money(valuation.head())

```

Out-of-fold R2 (log scale): 0.599

Median CBA discount on rookie deals: \$508,978

	Player	Team	Pos	Age	rookie	G	MP	PTS	actual	expected	market	surplus
0	Tyus Jones	3TM	PG	29.0	0	67.0	14.6	3.0	\$2,449,421	\$4,372,610	\$4,372,610	\$1,923,189
1	Walter Clayton	2TM	PG	22.0	1	69.0	20.4	7.8	\$4,190,520	\$3,860,560	\$4,543,249	\$352,729
2	AJ Johnson	2TM	SG	21.0	1	48.0	9.5	3.3	\$3,237,120	\$3,865,161	\$4,066,116	\$828,996
3	CJ McCollum	2TM	PG	34.0	0	76.0	29.8	18.7	\$21,000,000	\$18,202,190	\$18,202,190	\$-2,797,810
4	Khris Middleton	2TM	SF	34.0	0	63.0	22.8	10.2	\$5,591,122	\$7,418,331	\$7,418,331	\$1,827,209

```

undervalued = valuation.sort_values("surplus", ascending=False).head(25).reset_index(drop=True)
money(undervalued)

```

	Player	Team	Pos	Age	rookie	G	MP	PTS	actual	expected	market	surplus
0	LeBron James	LAL	SF	41.0	0	60.0	33.2	20.9	\$3,876,529	\$44,482,193	\$44,482,193	\$40,605,66
1	Keyonte George	UTA	PG	22.0	1	54.0	33.1	23.6	\$6,563,925	\$31,369,386	\$39,391,229	\$32,827,30
2	Victor Wembanyama	SAS	C	22.0	1	64.0	29.2	25.0	\$16,868,246	\$22,388,606	\$42,257,798	\$25,389,55
3	Cooper Flagg	DAL	SF	19.0	1	70.0	33.5	21.0	\$14,517,480	\$21,729,104	\$39,101,461	\$24,583,98
4	Stephon Castle	SAS	PG	21.0	1	68.0	30.0	16.7	\$10,015,920	\$22,171,877	\$34,483,232	\$24,467,31
5	Deni Avdija	POR	SF	25.0	0	66.0	33.3	24.2	\$13,125,000	\$35,722,886	\$35,722,886	\$22,597,88
6	Kevin Porter Jr.	MIL	PG	25.0	0	38.0	33.2	17.4	\$5,390,700	\$27,668,869	\$27,668,869	\$22,278,16
7	Ryan Rollins	MIL	PG	23.0	1	74.0	32.1	17.3	\$4,000,000	\$18,634,802	\$21,959,299	\$17,959,29
8	Nikola Vučević	2TM	C	35.0	0	64.0	28.4	15.1	\$2,449,421	\$19,939,899	\$19,939,899	\$17,490,47
9	Kyshawn George	WAS	SF	22.0	1	48.0	29.0	14.8	\$3,108,000	\$11,682,889	\$19,284,546	\$16,176,54
10	Nickeil Alexander-Walker	ATL	SG	27.0	0	78.0	33.4	20.8	\$14,403,710	\$29,718,411	\$29,718,411	\$15,314,70
11	Brandon Miller	CHO	SF	23.0	1	65.0	30.3	20.2	\$15,104,626	\$22,736,144	\$29,405,421	\$14,300,79
12	Jaime Jaquez Jr.	MIA	SF	24.0	1	75.0	28.3	15.4	\$5,939,141	\$19,360,245	\$20,108,751	\$14,169,61
13	Amen Thompson	HOU	PG	23.0	1	79.0	37.4	18.3	\$12,258,609	\$18,456,629	\$24,558,467	\$12,299,85
14	Saddiq Bey	NOP	SF	26.0	0	72.0	31.2	17.7	\$6,440,678	\$18,657,065	\$18,657,065	\$12,216,38
15	Norman Powell	MIA	SG	32.0	0	58.0	29.6	21.7	\$21,500,000	\$33,649,945	\$33,649,945	\$12,149,94
16	Neemias Queta	BOS	C	26.0	0	76.0	25.3	10.2	\$2,667,944	\$14,642,618	\$14,642,618	\$11,974,67
17	Payton Pritchard	BOS	PG	28.0	0	79.0	32.4	17.0	\$7,767,857	\$19,516,407	\$19,516,407	\$11,748,55
18	Kon Knueppel	CHO	SF	20.0	1	81.0	31.5	18.5	\$10,516,560	\$14,265,333	\$22,165,662	\$11,649,10
19	Kelly Oubre Jr.	PHI	SF	30.0	0	50.0	31.5	14.1	\$8,050,000	\$18,906,897	\$18,906,897	\$10,856,89
20	De'Anthony Melton	GSW	PG	27.0	0	49.0	23.0	12.3	\$3,451,779	\$13,843,376	\$13,843,376	\$10,391,59

	Player	Team	Pos	Age	rookie	G	MP	PTS	actual	expected	market	surplus
21	Moussa Diabaté	CHO	C	24.0	1	73.0	26.0	7.9	\$2,461,462	\$11,840,980	\$12,586,183	\$10,124,721
22	Shaedon Sharpe	POR	SG	22.0	1	50.0	29.4	20.8	\$20,089,287	\$19,634,654	\$30,096,837	\$10,007,551
23	Julius Randle	MIN	PF	31.0	0	79.0	33.0	21.1	\$33,333,334	\$42,529,972	\$42,529,972	\$9,196,638
24	Marcus Smart	LAL	SG	31.0	0	62.0	28.5	9.3	\$6,064,000	\$14,997,694	\$14,997,694	\$8,933,694

The **rookie** column now labels the first distortion rather than leaving you to infer it.

Rookie scale players will still top this list, and now that is the correct answer rather than an artifact. Surplus is measured against open market pricing, so a productive 22-year-old genuinely is paid far below what he would command. The difference is that the number is now a real estimate of that gap instead of a blend of two pricing regimes. It is useful for extension planning and for valuing a trade asset. It is not a signing opportunity, because his team controls him.

Veterans on minimum contracts stay in the sample deliberately. The minimum is a real market price for most of the players earning it, so removing them would have thrown away a legitimate part of the salary structure and truncated the target from below. The cost is that the handful who took a discount to join a contender do look underpaid, and to the model that is indistinguishable from being underpaid on merit.

The filtered table further down separates the signable names from the rest.

```
overvalued = valuation.sort_values("surplus").head(25).reset_index(drop=True)
money(overvalued)
```

	Player	Team	Pos	Age	rookie	G	MP	PTS	actual	expected	market	surplus
0	Stephen Curry	GSW	PG	37.0	0	43.0	30.9	26.6	\$62,587,158	\$24,946,301	\$24,946,301	\$-37,640,857
1	Jimmy Butler	GSW	SF	36.0	0	38.0	31.1	20.0	\$56,832,773	\$27,012,099	\$27,012,099	\$-29,820,674
2	Zach LaVine	SAC	SG	30.0	0	39.0	31.4	19.2	\$48,967,380	\$20,757,996	\$20,757,996	\$-28,209,384
3	Paul George	PHI	PF	35.0	0	37.0	30.7	17.3	\$54,126,380	\$27,479,847	\$27,479,847	\$-26,646,533
4	Evan Mobley	CLE	PF	24.0	1	65.0	31.9	18.2	\$50,105,628	\$23,921,079	\$24,925,944	\$-25,179,684
5	Jaren Jackson Jr.	2TM	C	26.0	0	48.0	30.3	19.4	\$49,000,000	\$24,174,772	\$24,174,772	\$-24,825,228
6	Jordan Poole	NOP	PG	26.0	0	39.0	23.9	13.4	\$34,044,642	\$10,270,643	\$10,270,643	\$-23,773,999

	Player	Team	Pos	Age	rookie	G	MP	PTS	actual	expected	market	surplus
7	Jalen Green	PHO	SG	23.0	1	32.0	25.9	17.8	\$36,251,166	\$9,308,852	\$12,517,306	\$-23,733,860
8	Anthony Davis	DAL	PF	32.0	0	20.0	31.3	20.4	\$58,456,566	\$36,130,742	\$36,130,742	\$-22,325,824
9	OG Anunoby	NYK	PF	28.0	0	67.0	33.2	16.7	\$42,500,000	\$20,481,478	\$20,481,478	\$-22,018,522
10	Karl-Anthony Towns	NYK	C	30.0	0	75.0	31.0	20.1	\$57,078,728	\$35,073,662	\$35,073,662	\$-22,005,066
11	LaMelo Ball	CHO	PG	24.0	1	72.0	28.0	20.1	\$40,770,520	\$18,687,266	\$18,926,561	\$-21,843,959
12	Devin Booker	PHO	SG	29.0	0	64.0	33.5	26.1	\$57,078,728	\$35,740,742	\$35,740,742	\$-21,337,986
13	Jalen Suggs	ORL	PG	24.0	1	57.0	27.6	13.8	\$32,400,000	\$10,883,608	\$11,234,146	\$-21,165,854
14	Jamal Murray	DEN	PG	28.0	0	75.0	35.4	25.4	\$50,105,628	\$30,462,182	\$30,462,182	\$-19,643,446
15	Kevin Durant	HOU	SF	37.0	0	78.0	36.4	26.0	\$43,902,439	\$24,316,651	\$24,316,651	\$-19,585,788
16	Darius Garland	2TM	PG	26.0	0	45.0	29.9	18.8	\$42,166,510	\$22,859,332	\$22,859,332	\$-19,307,178
17	Jrue Holiday	POR	PG	35.0	0	53.0	29.4	16.3	\$34,800,000	\$15,907,756	\$15,907,756	\$-18,892,244
18	Ja Morant	MEM	PG	26.0	0	20.0	28.5	19.5	\$42,166,510	\$23,957,288	\$23,957,288	\$-18,209,222
19	De'Aaron Fox	SAS	PG	28.0	0	72.0	31.0	18.6	\$49,800,000	\$31,695,541	\$31,695,541	\$-18,104,459
20	Chet Holmgren	OKC	PF	23.0	1	69.0	28.9	17.1	\$41,500,000	\$15,118,522	\$24,394,974	\$-17,105,026
21	Myles Turner	MIL	C	29.0	0	71.0	26.9	11.9	\$26,584,164	\$9,732,343	\$9,732,343	\$-16,851,821
22	Kentavious Caldwell-Pope	MEM	SG	32.0	0	51.0	21.3	8.4	\$21,621,500	\$5,596,025	\$5,596,025	\$-16,025,475
23	Aaron Gordon	DEN	PF	30.0	0	36.0	27.9	16.2	\$33,658,037	\$17,971,025	\$17,971,025	\$-15,687,012
24	Mikal Bridges	NYK	SF	29.0	0	82.0	32.8	14.4	\$33,482,145	\$17,859,731	\$17,859,731	\$-15,622,414

This tail is dominated by max and near-max contracts, and it is largely an artifact.

The model cannot predict above the highest salary it trained on, and a max player is at that ceiling by definition, so his expected salary is pulled down and his surplus is forced negative. It also cannot see

the things max contracts actually pay for: defensive impact, gravity, availability in the playoffs, and the fact that star production is worth more per unit than replacement production because roster spots are scarce.

The genuine signal in this tail is the players on large contracts who are **not** stars. A \$30M salary attached to modest volume is a real finding. A \$55M salary attached to a leading scorer is the ceiling effect.

```
# Rookie scale players are excluded because their team controls them, not because the
# model mistrusts the number. The salary floor is a noise filter for the compression
# bias rather than a signability rule: cheap players are predicted upward mechanically,
# so the very bottom of the pay scale produces spurious surplus. Lower or drop it to
# surface veterans on minimum deals, and read those names with extra scepticism.
MIN_TIER = 5_000_000

targets = (valuation[(valuation["surplus"] > 0)
                      & (valuation["rookie"] == 0)
                      & (valuation["actual"] >= MIN_TIER)])
          .sort_values("surplus", ascending=False)
          .head(20)
          .reset_index(drop=True))

print(f"{len(targets)} signable candidates "
      f"({(valuation['rookie'] == 1).sum()} rookie-scale players excluded)")
money(targets)
```

20 signable candidates (159 rookie-scale players excluded)

	Player	Team	Pos	Age	rookie	G	MP	PTS	actual	expected	market	surplus
0	Deni Avdija	POR	SF	25.0	0	66.0	33.3	24.2	\$13,125,000	\$35,722,886	\$35,722,886	\$22,597,886
1	Kevin Porter Jr.	MIL	PG	25.0	0	38.0	33.2	17.4	\$5,390,700	\$27,668,869	\$27,668,869	\$22,278,169
2	Nickeil Alexander-Walker	ATL	SG	27.0	0	78.0	33.4	20.8	\$14,403,710	\$29,718,411	\$29,718,411	\$15,314,701
3	Saddiq Bey	NOP	SF	26.0	0	72.0	31.2	17.7	\$6,440,678	\$18,657,065	\$18,657,065	\$12,216,387
4	Norman Powell	MIA	SG	32.0	0	58.0	29.6	21.7	\$21,500,000	\$33,649,945	\$33,649,945	\$12,149,945
5	Payton Pritchard	BOS	PG	28.0	0	79.0	32.4	17.0	\$7,767,857	\$19,516,407	\$19,516,407	\$11,748,550
6	Kelly Oubre Jr.	PHI	SF	30.0	0	50.0	31.5	14.1	\$8,050,000	\$18,906,897	\$18,906,897	\$10,856,897
7	Julius Randle	MIN	PF	31.0	0	79.0	33.0	21.1	\$33,333,334	\$42,529,972	\$42,529,972	\$9,196,638

	Player	Team	Pos	Age	rookie	G	MP	PTS	actual	expected	market	surplus
8	Marcus Smart	LAL	SG	31.0	0	62.0	28.5	9.3	\$6,064,000	\$14,997,694	\$14,997,694	\$8,933,694
9	Collin Gillespie	PHO	PG	26.0	0	80.0	28.5	12.7	\$10,714,286	\$19,254,224	\$19,254,224	\$8,539,938
10	Anfernee Simons	2TM	SG	26.0	0	55.0	24.9	14.3	\$6,000,000	\$14,336,362	\$14,336,362	\$8,336,362
11	Donte DiVincenzo	MIN	SG	29.0	0	82.0	30.4	12.2	\$12,535,000	\$20,060,635	\$20,060,635	\$7,525,635
12	Deandre Ayton	LAL	C	27.0	0	72.0	27.2	12.5	\$8,104,000	\$15,620,736	\$15,620,736	\$7,516,736
13	Shai Gilgeous-Alexander	OKC	PG	27.0	0	68.0	33.2	31.1	\$40,806,150	\$47,958,852	\$47,958,852	\$7,152,702
14	Andrew Nembhard	IND	PG	26.0	0	57.0	31.3	16.9	\$19,550,160	\$25,989,572	\$25,989,572	\$6,439,412
15	Dillon Brooks	PHO	SF	30.0	0	56.0	30.4	20.2	\$19,992,727	\$26,202,905	\$26,202,905	\$6,210,178
16	Royce O'Neale	PHO	SF	32.0	0	78.0	28.4	9.8	\$10,875,000	\$16,855,029	\$16,855,029	\$5,980,029
17	Kris Dunn	LAC	PG	31.0	0	82.0	27.2	7.3	\$5,684,800	\$10,557,208	\$10,557,208	\$4,872,408
18	Aaron Nesmith	IND	SF	26.0	0	45.0	29.7	13.8	\$11,000,000	\$15,872,145	\$15,872,145	\$4,872,145
19	Grayson Allen	PHO	SG	30.0	0	51.0	28.8	16.5	\$18,125,000	\$22,723,832	\$22,723,832	\$4,598,832

This is the list that answers the original question.

Filtering to players above the minimum salary band and past typical rookie-scale age removes the two structural distortions and leaves players whose current contract was actually negotiated on the open market. A positive surplus here means a team is paying below the going rate for that statistical profile, which is what makes the contract attractive to acquire and what makes the player likely to be underpriced at his next negotiation.

The rookie flag is an age proxy. Rookie scale runs four years from the draft, which usually spans ages 19 to 24, but it misclassifies undrafted young players on minimum deals and the occasional late entrant. Years of experience is the correct field and it is not on the per-game page. If you pull the player pages, swap it in and the flag stops being an approximation.

```
under = valuation.nlargest(10, "surplus")
over = valuation.nsmallest(10, "surplus")

fig, ax = plt.subplots(figsize=(12, 7.5))
```

```

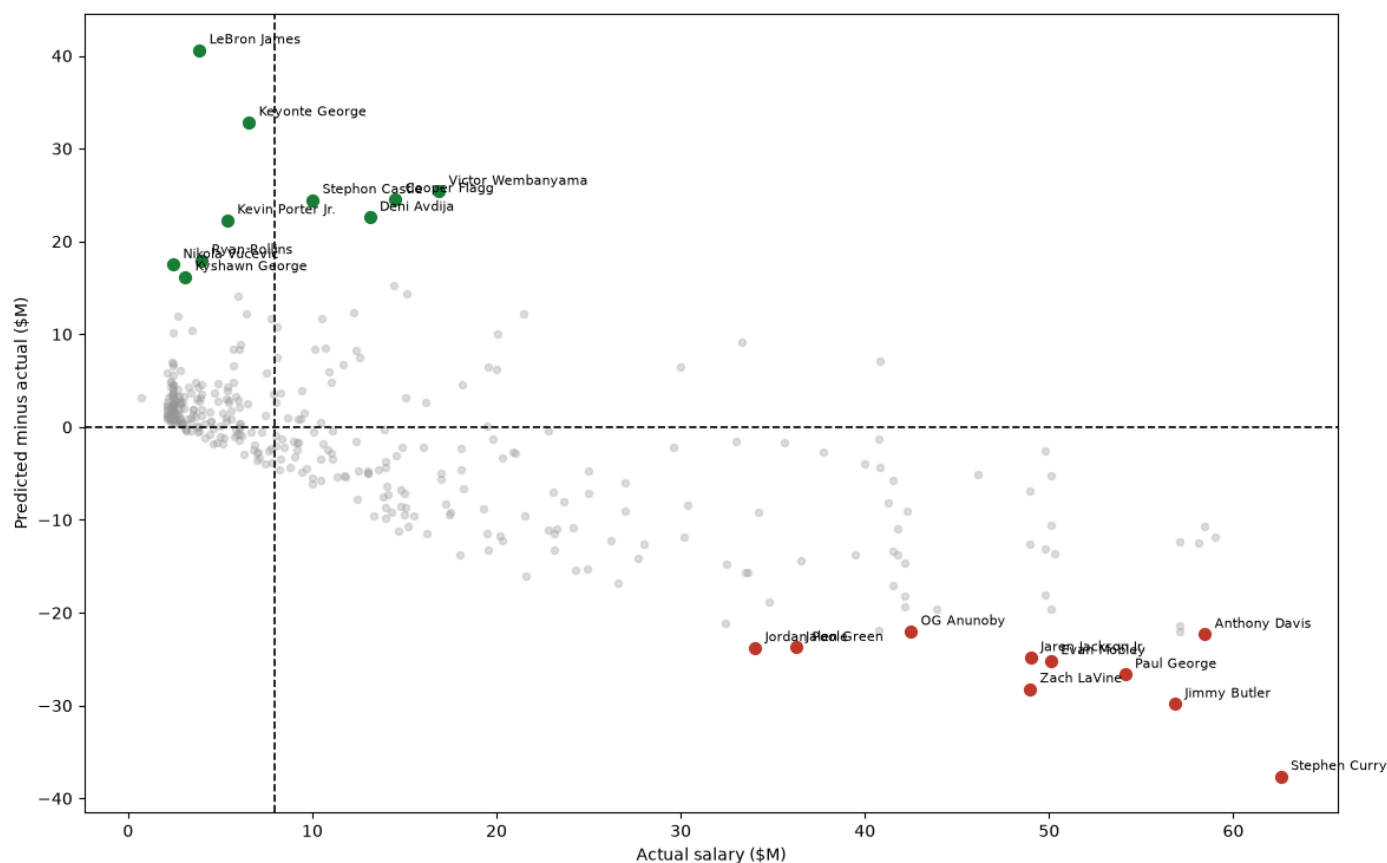
ax.scatter(valuation["actual"] / 1e6, valuation["surplus"] / 1e6,
           s=18, alpha=0.3, color="#999999")
ax.scatter(under["actual"] / 1e6, under["surplus"] / 1e6, s=50, color="#1a7f37")
ax.scatter(over["actual"] / 1e6, over["surplus"] / 1e6, s=50, color="#c0392b")

ax.axhline(0, ls="--", lw=1.2, color="black")
ax.axvline(valuation["actual"].median() / 1e6, ls="--", lw=1.2, color="black")

for _, r in pd.concat([under, over]).iterrows():
    ax.annotate(r["Player"], (r["actual"] / 1e6, r["surplus"] / 1e6),
               xytext=(6, 4), textcoords="offset points", fontsize=8)

ax.set_xlabel("Actual salary ($M)")
ax.set_ylabel("Predicted minus actual ($M)")
plt.tight_layout()
plt.show()

```



The downward drift from upper left to lower right is the compression effect made visible across the whole league, not a basketball finding: cheap players trend positive and expensive players trend negative almost mechanically.

Because of that, the interesting region is not the extreme upper left. It is the upper right, past the vertical median line and above zero. A player who is already well paid and still scores positive surplus is one the model rates highly despite the pull toward the mean, which makes him a more robust signal than anyone clustered at the cheap end.

Conclusion

Volume and role, not efficiency. Points, shots, and minutes correlate with salary between roughly 0.6 and 0.75, while true shooting, effective field goal percentage and the other rate statistics sit near zero. Both the marginal correlations and the fitted importances agree. Any search for bargains built on this data is therefore a search for players absorbing a starter's workload on a reserve's contract.

Roughly 60 percent of the variance in log salary, with a mean absolute error in the millions. That supports ranking and screening. It does not support pricing an individual contract, and the output should not be quoted as a number a player is worth.

Three biases that shape the results and do not come from basketball.

1. Regression toward the mean. Cheap players are predicted upward, expensive players downward. The undervalued list is biased toward low salaries before any evaluation happens.
2. No extrapolation. Trees cannot predict beyond the training range, so max contract players are structurally forced into the overvalued tail.
3. CBA-fixed contracts. Rookie scale is now a feature rather than a hidden confound, so the model no longer averages negotiated and scale pricing into one curve, and every player gets a market rate estimate. Those players still dominate the raw undervalued list, correctly, but they are unavailable. Minimum deals remain in the sample because the minimum is a genuine market price for most of the players on it.

Use the filtered candidate table, not the raw undervalued list. Excluding rookie-scale players leaves those a rival can actually pursue, and a positive surplus there is a real pricing gap rather than an accounting one. Use the raw list separately, for extension planning and trade valuation, where the CBA discount is the point rather than the problem. Within that set, compare inside position groups, since the residuals by position show the box score treats frontcourt value poorly. Weight candidates in the upper right of the scatter, where a positive surplus survives the pull toward the mean.

This model finds players who are cheap relative to their statistical production. Whether they are cheap relative to their contribution to winning is a different question that a box score cannot answer, and the gap between those two is exactly where front offices disagree with each other. Treat the output as a screen that generates names for film and tracking review, not as a conclusion.

The learning curve flattens well before the full sample, so more players will not help. Better features will: years of experience and contract signing date to replace the proxies above, tracking data for the defensive and off-ball value the box score misses, and injury history to explain the availability discount that shows up as unexplained residual.